

gbrainに学ぶ、Agentの記憶基盤とKnowledge Graphの作り方

koki takeishi 2026-05-10T02:34:06+00:00

PostgreSQL

knowledgegraph

AIエージェント

agetmemory

tech

<https://zenn.dev/headwaters/articles/8bc4e8c3119fa3>

前書き

Y CombinatorのCEOが作成した、gbrainというOSSがとんでもなさそうなので、CODEXと共に gbrain というリポジトリを読み、Agentの記憶基盤がどのように作られているか、特にKnowledge Graphをどう使っているかを整理します。(2026/5/10 JST)



Meta-Meta-Prompting: The Secret to Making AI Agents Work

対象リポジトリ

<https://github.com/garrytan/gbrain>

!

この記事では書かないこと

Agentの記憶基盤の技術調査がメインだったので、このOSSの価値や重要な点は実際のリポジトリを見たり、イン

スツールして動かすことをお勧めします。現状のITエンジニアが感じているカスタムAgentやSkillsの再利用、それを業務効率化に繋げるためのヒントにもなるサービスだと思います。

このサービスの背景

AI Agentを作っていると、Agentの「記憶」をどう扱うかという問題にぶつかります。

単純なRAGだけでも、ドキュメントを検索して文脈に入れることはできます。しかし、現実の業務で欲しくなるのは単なる類似文書検索だけではありません。

たとえば、次のような質問です。

- Alice はどの会社で働いているのか
- Bob はどの会社に投資したのか
- この会議には誰が参加したのか
- この会社と自分の接点はどこにあるのか
- ある関数を呼んでいるコードはどこか

こうした質問は、ベクトル検索だけだと安定しにくいです。文章として似ているかどうかではなく、「人」「会社」「会議」「投資」「所属」といった関係そのものを辿りたいからです。

gbrainとは何か

gbrain は、AI Agent に「brain」を持たせるための CLI / MCP server / DB runtime です。

Markdown、会議メモ、コードリポジトリ、記事、画像などを取り込み、Postgres または PGLite に保存します。その上で、Agent から MCP tool として検索・取得・更新・Graph traversal できるようにします。

大きく見ると、構成は次のようになっています。

面白いのは、LLM にすべてを任せるのではなく、かなりの部分を deterministic な TypeScript + SQL runtime として実装している点です。

Agent は skills を読んで判断しますが、保存、検索、Graph extraction、MCP dispatch、権限、DB schema は明確なコードとして実装されています。

中心となるデータモデル

gbrain の中心は `pages` と `content_chunks` です。

データ層だけを見ると、次のような関係になっています。

`pages` は、ひとつの brain page を表します。

```
pages
  source_id
  slug
  type
  page_kind
  title
  compiled_truth
  timeline
  frontmatter
  emotional_weight
  effective_date
```

`compiled_truth` は、その page についての現在の理解をまとめた部分です。 `timeline` は時系列の証跡です。gbrain はこの2つを分けて扱います。

検索の単位は `content_chunks` です。

```
content_chunks
  page_id
  chunk_index
  chunk_text
  chunk_source
  embedding
  search_vector
  language
  symbol_name
  symbol_name_qualified
  modality
  embedding_image
```

ここに text embedding、Postgres full-text search 用の `tsvector`、コード用の symbol metadata、画像用 embedding などが入ります。

そして Knowledge Graph の中心が `links` です。

```
links
  from_page_id
  to_page_id
  link_type
  context
  link_source
  origin_page_id
  origin_field
  resolution_type
```

`links` は page 間の typed edge です。たとえば、次のような関係を表します。

```
people/alice --works_at--> companies/acme
people/bob   --invested_in--> companies/acme
people/carol --attended--> meetings/weekly-sync
people/dave  --founded--> companies/startup-x
```

RAG の文書集合に「関係」を足している、と見るとわかりやすいです。

記憶層のアーキテクチャ

gbrain の記憶層は、単一の vector index ではありません。短期的な事実、長期的な page、検索 chunk、typed graph、時系列を重ねて扱います。

この図のポイントは、記憶が用途別に分かれていることです。

- `facts` : 会話などから抽出される短期/作業記憶
- `pages` : 現在の理解をまとめた長期記憶
- `timeline_entries` : いつ何が起きたかという時系列記憶
- `content_chunks` : 検索用の retrieval 記憶
- `links` : 人・会社・会議などの関係記憶
- `code_edges_*` : コード構造の記憶
- `takes` : fact/take/bet/hunch のような claim 記憶

つまり gbrain は「全部を embedding に入れる」のではなく、記憶の種類ごとに保存先と検索方法を変えています。

Knowledge Graphはどう作られるのか

Knowledge Graph の生成ロジックは主に `src/core/link-extraction.ts` にあります。

gbrain は、LLM を呼ばずに link を抽出します。README でも "zero LLM calls" と説明されており、ここが設計上かなり重要です。

1. Markdown link / wikilink から抽出する

まず、Markdown 内のリンクを見ます。

```
[Alice](people/alice)
[Alice](../people/alice.md)
[[people/alice]]
[[people/alice|Alice]]
[[source-id:people/alice|Alice]]
```

対象になるディレクトリは whitelist 方式です。

```
people
companies
meetings
concepts
deal
projects
source
media
...
```

つまり、すべてのリンクを graph edge にするのではなく、「entity として扱うディレクトリに向いているリンク」を抽出します。

また、コードブロックや inline code は抽出前に除外されます。コード例の中に `people/foo` のような文字列が出てきても、それは実世界の関係ではないからです。

2. bare slug も拾う

Markdown link になっていなくても、本文中に `people/alice` や `companies/acme` のような slug が直接出てくれば候補になります。

```
Aliceについては people/alice を参照
```

このような記述も edge 化できます。

3. frontmatterから構造化edgeを作る

特に面白いのが frontmatter です。

gbrain は YAML frontmatter の field を見て、typed edge を生成します。

たとえば person page に次のような frontmatter があるとします。

```
---
type: person
company: Acme
founded:
  - Startup X
---
```

これは概念的には次の edge になります。

```
people/<person> --works_at--> companies/acme
people/<person> --founded--> companies/startup-x
```

company page では向きが逆になることがあります。

```
---
type: company
key_people:
  - Alice
investors:
  - Bob
---
```

この場合、保存される edge は次の向きです。

```
people/alice --works_at--> companies/<company>
people/bob --invested_in--> companies/<company>
```

company から person へ edge を張るのではなく、関係の意味として自然な「主語 -> 目的語」の向きに寄せています。

これは graph query を書くときに効きます。

```
gbrain graph-query companies/acme --type works_at --direction in
```

このようにすれば、「Acme で働いている人」を incoming edge として取得できます。

link_typeはどう決まるのか

Markdown の文脈から作る edge では、周辺テキストを見て link_type を推定します。

推定は LLM ではなく正規表現です。

優先順位はおおむね次のようになっています。

```
founded
> invested_in
> advises
> works_at
> page-role prior
> mentions
```

たとえば次のような表現を拾います。

表現	link_type
founded , co-founded , founder of	founded
invested in , led the seed , portfolio company	invested_in
advisor to , advisory board , consults for	advises
CEO of , engineer at , worked at	works_at
meeting page から person への参照	attended

この方式は LLM 抽出より柔軟性では劣りますが、再現性・速度・コストの面で強いです。

特に、大量ページを backfill するときに「毎回 LLM を呼ぶ」設計だと現実的に厳しくなります。gbrain は gbrain extract links --source db で既存の DB 内 pages を走査し、deterministic に graph を作れます。

stale edgeをどう消すか

Knowledge Graph でよくある問題は、edge が追加される一方で古い関係が残り続けることです。

gbrain は put_page 後の auto-link で、既存 edge と現在の本文/frontmatter から抽出した edge を比較し、不要になった edge を削除します。

ここで言う extraction と reconciliation は、次の2段階です。

- extraction: 現在の page から「あるべき edge の候補」を取り出す
- reconciliation: DB に残っている edge と比較し、差分だけを反映する

たとえば、最初に次のような meeting page があったとします。

```
---
type: meeting
attendees:
  - Alice
---
```

Acme の採用計画について話した。

この page からは、概念的に次の edge が抽出されます。

```
people/alice --attended--> meetings/acme-hiring
meetings/acme-hiring --mentions--> companies/acme
```

その後、meeting page を編集して attendee を Alice から Bob に変えたとします。

```
---
type: meeting
attendees:
  - Bob
---
```

Acme の採用計画について話した。

このとき、単に新しい edge を追加するだけだと、Alice の参加履歴が残ってしまいます。実際には Alice はもうこの page の attendee ではありません。

そこで gbrain は、現在の page からもう一度 edge を抽出し、DB にある既存 edge と比較します。

既存DB:

```
people/alice --attended--> meetings/acme-hiring
meetings/acme-hiring --mentions--> companies/acme
```

今回抽出:

```
people/bob --attended--> meetings/acme-hiring
meetings/acme-hiring --mentions--> companies/acme
```

差分:

```
remove: people/alice --attended--> meetings/acme-hiring
add:    people/bob --attended--> meetings/acme-hiring
keep:   meetings/acme-hiring --mentions--> companies/acme
```

この差分反映が reconciliation です。

処理の流れはこうです。

```
page write
-> importFromContent
-> extractPageLinks
-> 既存 links / backlinks を取得
-> 今回の候補と比較
-> 新しい edge を追加
-> 消えた edge を削除
```

ただし、すべての edge を雑に消すわけではありません。

links table には link_source と origin_page_id があります。

- markdown 由来
- frontmatter 由来
- manual 由来

この provenance を見て、reconciliation の対象を限定します。

たとえば manual に追加された edge は auto-link の編集で勝手に消されません。frontmatter 由来 edge も「その page の frontmatter が作ったもの」だけを消します。

Graph を運用するには、この provenance が重要です。そうしないと、ある page の編集が他の page の関係まで壊してしまいます。

つまりreconciliationとは、単に DELETEしてINSERTし直す のではなく、edgeの由来と作成元を見ながら、その pageが責任を持つedgeだけを更新するということです。

Graphは検索にどう効くのか

gbrain は graph を単なる可視化用データとして扱っていません。検索ランキングにも使います。

src/core/search/hybrid.ts では、検索結果の slug ごとに backlink count を取得し、score を増幅します。

```
score *= 1 + 0.05 * log(1 + backlink_count)
```

つまり、多くの page から参照される entity は検索で少し上がります。

これは PageRank ほど複雑なものではありませんが、「よく接続されている page は重要」という signal を retrieval に混ぜるには十分に実用的です。

検索 pipeline 全体は次のようになっています。

```
query
-> intent/detail auto detection
-> optional query expansion
-> keyword search
-> vector search
-> RRF fusion
-> cosine re-score
-> backlink / salience / recency boost
-> dedup
```

ポイントは、embedding が使えない場合でも keyword search と backlink boost は動くことです。Graph は vector search の補助ではなく、独立した ranking signal として使われています。

Graph traversalとしても使う

gbrain には `graph-query` があります。

```
gbrain graph-query people/alice --type attended --depth 2
gbrain graph-query companies/acme --type works_at --direction in
gbrain graph-query people/bob --type invested_in --depth 1
```

内部的には `traversePaths` が recursive CTE で graph を辿ります。

recursive CTE は、SQL の中で再帰的に行を増やしていく仕組みです。

普通の SQL は「この条件に一致する行を取る」という1段階の検索が得意です。一方、Graph traversal では「Alice が参加した会議」「その会議に参加した別の人」「その人が所属する会社」のように、edge を何段も辿りたくなります。

アプリケーション側でループしても実装できますが、gbrain は DB 側の recursive CTE を使って、`links` table を複数 hop 辿ります。

イメージはこうです。

```
start page
depth 0: people/alice
  |
  | attended
  v
depth 1: meetings/weekly-sync
  |
  | attended
  v
depth 2: people/bob
```

SQL の中では、おおむね次のような考え方になります。

```

WITH RECURSIVE walk AS (
  -- まず起点 page を入れる
  SELECT id, slug, 0 AS depth, ARRAY[id] AS visited
  FROM pages
  WHERE slug = 'people/alice'

  UNION ALL

  -- 直前に見つかった page から links を1段辿る
  SELECT next_page.id,
         next_page.slug,
         walk.depth + 1,
         walk.visited || next_page.id
  FROM walk
  JOIN links ON links.from_page_id = walk.id
  JOIN pages AS next_page ON next_page.id = links.to_page_id
  WHERE walk.depth < 2
         AND NOT (next_page.id = ANY(walk.visited))
)
SELECT * FROM walk;

```

重要なのは `visited` です。

Graphには循環してしまう問題があります。

```
people/alice -> meetings/a -> people/bob -> meetings/a -> ...
```

何も考えずに再帰すると、同じ node をぐるぐる辿ってしまいます。そこで `visited` 配列に「すでに通った page_id」を入れておき、訪問済みの page は辿らないようにします。

また、`--type works_at` のような link type filter や、`--direction in|out|both` の direction control も traversal の条件に入ります。

`traverse_graph` operation ではdepthが最大10にclampされます。特にMCPのような外部 tool surfaceから呼ばれる場合、Graph traversal は fan-out しやすいので、深さに上限を置くのは DoS対策として自然です。

▶ DoS対策

この Graph traversal は、ベクトル検索ではなく構造検索です。

「Acmeに関連する文章」ではなく、「Acmeにworks_atで入ってくる人」を取りに行けます。

コードにもGraphがある

gbrain は人・会社・会議の graph だけでなく、コード用の graph も持っています。

Schema には次の2つがあります。

```
code_edges_chunk
  resolved edge
  from_chunk_id -> to_chunk_id

code_edges_symbol
  unresolved symbol reference
  from_chunk_id -> to_symbol_qualified
```

resolved と unresolved を分けているのが良い設計です。import order の都合で、参照先 symbol の chunk がまだ存在しない場合でも edge を失わずに保存できます。

CLI も用意されています。

```
gbrain code-def BrainEngine
gbrain code-refs BrainEngine
gbrain code-callers searchKeyword
gbrain code-callees searchKeyword
```

さらに `query` では `--near-symbol` と `--walk-depth` を使えます。

```
gbrain query "search rankingの処理を知りたい" \
  --near-symbol BrainEngine.searchKeyword \
  --walk-depth 2
```

この場合、通常の検索結果を anchor にして、code graph を最大2 hop 辿り、構造的に近い chunk を検索結果に追加します。

これはコード調査用 Agent にはかなり有効です。単なる grep や embedding ではなく、「この関数の呼び出し元」「この symbol の周辺」を retrieval に入れられるからです。

multi-source設計

gbrain には brain と source という2つの軸があります。

- brain: DB
- source: DB 内の論理 repo / namespace

`pages` は `source_id + slug` で一意です。

同じ `topics/ai` という slug が、`wiki` source と `gstack` source に存在しても構いません。

このため、link 作成時にも source-awareness が必要です。`addLinksBatch` では `from_source_id`, `to_source_id`, `origin_source_id` を渡せるようになっており、同じ slug が複数 source にある場合の fan-out を避けます。

また source には federation 設定があります。

- `federated=true` : default search に参加
- `federated=false` : 明示指定された時だけ検索

個人 brain に複数 repo を入れる、あるいはチーム brain と個人 brain を分ける、といった運用を想定していることがわかります。

セキュリティ上の面白い点

Knowledge Graph は検索ランキングに影響します。

ということは、攻撃者が arbitrary link を植えられると、検索結果を操作できる可能性があります。

gbrain はこの点をコードコメントでも明示しており、remote MCP の `put_page` では通常 auto-link を skip します。

```
remote MCP write
-> auto-link skipped

local CLI write
-> auto-link enabled

trusted workspace subagent
-> allow-list付きで auto-link enabled
```

Graph は単なるメタデータではなく、retrieval integrity に影響するデータです。ここを trust boundary として扱っているのは実用的です。

gbrainの設計から学べること

gbrain の Knowledge Graph 設計から学べること。

1. RAGにGraphを足すと、質問の種類が増える

ベクトル検索は「似ている文章」を探すのが得意です。

Graph は「関係」を辿るのが得意です。

Agentの記憶基盤では、この2つは競合ではなく補完関係です。

2. すべてをLLM抽出にしなくてよい

gbrain の page graph extraction は基本的に deterministic です。

Markdown link、wikilink、frontmatter、regex。

再現性・コスト・backfillのしやすさを考えると強い選択です。

3. edgeのprovenanceは必須

Graph は作るより保つほうが難しいです。

`link_source`, `origin_page_id`, `origin_field` を持つことで、どの edge を誰が作ったか、どの page の編集で消してよいかを判断できます。

4. Graphをranking signalにする

Graph traversal のためだけでなく、検索 ranking にも backlink count を使っています。

Graph を「検索結果の並びを改善するsignal」として使うのは、実装コストの割に効果が出やすいアプローチです。

5. コードGraphとKnowledge Graphを分ける

人・会社・会議の関係と、コードの call graph は性質が違います。

gbrain は `links` と `code_edges_*` を分けています。これは良い分離です。一方で、検索体験としては `query -near-symbol --walk-depth` のように統合されています。

逆に注意が必要な点

もちろん限界もあります。

まず、link type inference は正規表現に依存します。英語の特定表現には強いですが、日本語の「A社に所属」「Bに出資」「Cを創業」といった表現は、そのままでは拾いにくいはずです。

また、Graph query は自然言語質問を自動で graph traversal plan に変換しているわけではありません。 `graph-query` や MCP tool をどう使うかは、skills や agent workflow 側の責務です。

さらに、backlink boost は軽量の inbound count boost であり、本格的な PageRank ではありません。とはいえ、Agent memory の実用上はこのくらい単純なほうが運用しやすいとも言えます。

まとめ

gbrainは、Agentの記憶基盤を考えるうえでかなり参考になるリポジトリです。

特にKnowledge Graphまわりでは、次の設計が印象的でした。

- page 間の関係を `links` table に typed edge として保存する
- Markdown / wikilink / frontmatter から deterministic に edge を作る
- `works_at`, `invested_in`, `attended`, `founded` など実用的な link type を持つ

- edge provenance を保存し、編集時に stale edge を消す
- Graph traversal だけでなく backlink boost として検索rankingにも使う
- コード用には別の `code_edges_*` graph を持ち、two-pass retrievalに使う
- remote MCP では auto-link を制限し、retrieval integrityを守る

RAGを作るとき、つい embedding と chunking に目が行きます。しかし、Agent が本当に欲しいのは「似ている文章」だけではありません。

誰が、どこに所属し、何に投資し、どの会議に参加し、どのコードがどのコードを呼んでいるのか。

そうした関係を扱うには、Knowledge Graph が必要になります。

gbrainは、そのGraphを大げさなgraph databaseではなく、Postgresのtableとrecursive CTE、そして丁寧な extraction/reconciliationで実装しています。

Agentの記憶基盤を作るなら、「vector search + keyword search + typed graph」の組み合わせはかなり現実的な設計だと思います。